

BUILD-SUPABASE.md

Partie C.9 — Fonctions SQL

Version : V1

Statut : Référence officielle des fonctions PostgreSQL

1. Objectif

Cette section définit les règles officielles de conception, de développement et d'utilisation des fonctions SQL dans PostgreSQL.

Les fonctions SQL permettent :

- d'automatiser des traitements techniques ;
- de factoriser des requêtes réutilisables ;
- d'améliorer certaines performances ;
- d'exposer des traitements sécurisés.

Les fonctions SQL ne remplacent jamais les Domaines métier.

2. Principes directeurs

Toutes les fonctions SQL respectent les principes suivants :

- Technical First
- Domain First
- Single Responsibility
- Security by Design
- Documentation obligatoire

Une fonction SQL ne doit effectuer qu'une responsabilité clairement identifiée.

3. Responsabilités autorisées

Les fonctions SQL peuvent être utilisées pour :

- calculs techniques simples ;
- agrégations ;
- traitements de lecture ;
- normalisation de données ;

- validation technique ;
 - encapsulation de requêtes complexes ;
 - génération d'identifiants ou de métadonnées.
-

4. Responsabilités interdites

Les fonctions SQL ne doivent jamais :

- implémenter une règle métier complexe ;
- orchestrer plusieurs domaines ;
- remplacer un Use Case ;
- prendre une décision fonctionnelle ;
- gérer la navigation de l'application ;
- exécuter des workflows métier.

Ces responsabilités appartiennent exclusivement aux Domaines et aux APIs.

5. Langages autorisés

Les fonctions doivent être développées en :

- SQL
- PL/pgSQL

L'utilisation d'autres langages n'est autorisée qu'après validation architecturale.

6. Nommage

Les fonctions doivent respecter les conventions suivantes :

- minuscules ;
- `snake_case` ;
- nom explicite.

Exemples :

- `calculate_trust_score()`
- `refresh_dashboard_stats()`
- `normalize_phone_number()`

Les préfixes techniques peuvent être utilisés lorsque cela améliore la lisibilité.

7. Paramètres

Les paramètres doivent :

- être typés explicitement ;
- avoir des noms explicites ;
- être validés lorsque nécessaire.

Les paramètres génériques tels que `value` ou `data` sont à éviter.

8. Valeurs de retour

Une fonction doit retourner :

- une valeur simple ;
- un enregistrement ;
- un ensemble d'enregistrements ;
- un type composite.

Le type de retour doit être clairement documenté.

9. Sécurité

Les fonctions doivent respecter les politiques de sécurité de Vitala.

Lorsque `SECURITY DEFINER` est utilisé, cela doit être :

- justifié ;
- documenté ;
- audité.

Par défaut, les fonctions utilisent `SECURITY INVOKER`.

10. Gestion des erreurs

Les fonctions doivent gérer les erreurs de manière contrôlée.

Les messages retournés ne doivent jamais exposer :

- des informations sensibles ;
- des détails techniques inutiles ;
- la structure interne de la base de données.

Les erreurs détaillées sont consignées dans les journaux techniques.

11. Performances

Les fonctions doivent :

- éviter les traitements inutiles ;
- limiter les boucles coûteuses ;
- privilégier les opérations SQL optimisées.

Les fonctions fréquemment utilisées doivent être analysées avec `EXPLAIN ANALYZE`.

12. Réutilisabilité

Une fonction doit être conçue pour être réutilisable.

La duplication de logique SQL est interdite lorsqu'une fonction existante répond déjà au besoin.

13. Documentation

Chaque fonction doit être documentée avec :

- son objectif ;
 - ses paramètres ;
 - son type de retour ;
 - les tables utilisées ;
 - les éventuels effets secondaires ;
 - les règles de sécurité.
-

14. Évolution

Toute création ou modification d'une fonction doit :

- passer par une migration ;
 - être testée ;
 - être documentée ;
 - préserver la compatibilité lorsque cela est possible.
-

15. Tests

Les fonctions critiques doivent être testées pour vérifier :

- les cas nominaux ;
- les cas limites ;
- les erreurs ;
- les performances.

Les tests doivent être automatisés lorsque cela est pertinent.

16. Critères de conformité

Une fonction SQL est conforme lorsque :

- elle possède une responsabilité unique ;
 - elle ne contient pas de logique métier complexe ;
 - elle respecte les conventions de nommage ;
 - elle applique les règles de sécurité ;
 - elle est documentée ;
 - elle est testée.
-

17. Conclusion

Les fonctions SQL constituent un outil puissant pour centraliser les traitements techniques dans PostgreSQL.

Leur utilisation doit rester limitée aux responsabilités d'infrastructure afin de préserver une architecture Domain-Driven claire, évolutive et maintenable.